Q1: Create a custom type in ksqlDB called season_length that represents the type of the Two attributes before and after.

```
ksql> CREATE TYPE season_length AS STRUCT<season_id INT, episode_count INT>;

 Message
----------------------------------------------------------------------------------------------------------------
 Registered custom type with name 'SEASON_LENGTH' and SQL type STRUCT<`SEASON_ID` INTEGER, `EPISODE_COUNT` INTEGER>
----------------------------------------------------------------------------------------------------------------
ksql>  SHOW TYPES ;

 Type Name     | Schema
-------------------------------------------------------------
 SEASON_LENGTH | STRUCT<SEASON_ID INTEGER, EPISODE_COUNT INTEGER>
-------------------------------------------------------------
```

Q2 :Choose the most appropriate type of collection for titles and production_changestitles should be a TABLE:

answer:

A table represents the latest state of a dataset. For video titles, we only care about the current, up-to-date mapping of a Title ID to its Name, not the entire history of its typos or updates.production_changes should be a STREAM: A stream represents a continuous, immutable sequence of independent events over time. Each production change (like a budget update or season length change) is an isolated event that happened at a specific tim

Q3:

```
-------------------------------------------------------------
ksql> -- Create the titles Table
>CREATE TABLE titles (
>    id INT PRIMARY KEY,
>    title VARCHAR
>) WITH (
>    KAFKA_TOPIC = 'titles',
>    PARTITIONS = 4,
>    VALUE_FORMAT = 'JSON'
>);
>
>-- Create the production_changes Stream
>CREATE STREAM production_changes (
>    rowkey VARCHAR KEY,
>    title_id INT,
>    change_type VARCHAR,
>    before_season_length,
>    after_season_length,
>    created_at VARCHAR
>) WITH (
>    KAFKA_TOPIC = 'production_changes',
>    PARTITIONS = 4,
>    VALUE_FORMAT = 'JSON',
>    TIMESTAMP = 'created_at',
>    TIMESTAMP_FORMAT = 'yyyy-MM-dd HH:mm:ss'
>);

CREATE TABLE TITLES (ID INTEGER PRIMARY KEY, TITLE STRING) WITH (KAFKA_TOPIC='titles', KEY_FORMAT='KAFKA', PARTITIONS=4, VALUE_FORMAT='JSON');
 Message
---------------
 Table created
---------------

CREATE STREAM PRODUCTION_CHANGES (ROWKEY STRING KEY, TITLE_ID INTEGER, CHANGE_TYPE STRING, BEFORE STRUCT<SEASON_ID INTEGER, EPISODE_COUNT INTEGER>, AFTER STRUCT<SEASON_ID INTEGER, EPISODE_COUNT INTEGER>, CREATED_AT STRING) WITH (KAFKA_T
OPIC='production_changes', KEY_FORMAT='KAFKA', PARTITIONS=4, TIMESTAMP='created_at', TIMESTAMP_FORMAT='yyyy-MM-dd HH:mm:ss', VALUE_FORMAT='JSON');
 Message
---------------
 Stream created
---------------
ksql>
```

- the created tables

```
ksql> SHOW TABLES EXTENDED ;

Name                 : TITLES
Type                 : TABLE
Timestamp field      : Not set - using <ROWTIME>
Key format           : KAFKA
Value format         : JSON
Kafka topic          : titles (partitions: 4, replication: 1)
Statement            : CREATE TABLE TITLES (ID INTEGER PRIMARY KEY, TITLE STRING) WITH (KAFKA_TOPIC='titles', KEY_FORMAT='KAFKA', PARTITIONS=4, VALUE_FORMAT='JSON');

 Field | Type
----------------------------------------
 ID    | INTEGER          (primary key)
 TITLE | VARCHAR(STRING)
----------------------------------------

Local runtime statistics
------------------------


(Statistics of the local KSQL server interaction with the Kafka topic titles)
```

```
ksql> DESCRIBE titles ;

Name                 : TITLES
 Field | Type
----------------------------------------
 ID    | INTEGER          (primary key)
 TITLE | VARCHAR(STRING)
----------------------------------------
For runtime statistics and query details run: DESCRIBE <Stream,Table> EXTENDED;
ksql>
```

- the stream

```
ksql> SHOW STREAMS EXTENDED ;

Name                 : PRODUCTION_CHANGES
Type                 : STREAM
Timestamp field      : CREATED_AT
Key format           : KAFKA
Value format         : JSON
Kafka topic          : production_changes (partitions: 4, replication: 1)
Statement            : CREATE STREAM PRODUCTION_CHANGES (ROWKEY STRING KEY, TITLE_ID INTEGER, CHANGE_TYPE STRING, BEFORE STRUCT<SEASON_ID INTEGER, EPISODE_COUNT INTEGER>, AFTER STRUCT<SEASON_ID INTEGER, EPISODE_COUNT INTEGER>, CREATED_A
T STRING) WITH (KAFKA_TOPIC='production_changes', KEY_FORMAT='KAFKA', PARTITIONS=4, TIMESTAMP='created_at', TIMESTAMP_FORMAT='yyyy-MM-dd HH:mm:ss', VALUE_FORMAT='JSON');

 Field       | Type
-------------------------------------------------------------
 ROWKEY      | VARCHAR(STRING)  (key)
 TITLE_ID    | INTEGER
 CHANGE_TYPE | VARCHAR(STRING)
 BEFORE      | STRUCT<SEASON_ID INTEGER, EPISODE_COUNT INTEGER>
 AFTER       | STRUCT<SEASON_ID INTEGER, EPISODE_COUNT INTEGER>
 CREATED_AT  | VARCHAR(STRING)
-------------------------------------------------------------

Local runtime statistics
------------------------


(Statistics of the local KSQL server interaction with the Kafka topic production_changes)

Name                 : KSQL_PROCESSING_LOG
Type                 : STREAM
Timestamp field      : Not set - using <ROWTIME>
Key format           : KAFKA
Value format         : JSON
Kafka topic          : default_ksql_processing_log (partitions: 1, replication: 1)
Statement            : CREATE STREAM KSQL_PROCESSING_LOG (LOGGER STRING, LEVEL STRING, TIME BIGINT, MESSAGE STRUCT<TYPE INTEGER, DESERIALIZATIONERROR STRUCT<TARGET STRING, ERRORMESSAGE STRING, RECORDB64 STRING, CAUSE ARRAY<STRING>, `top
ic` STRING>, RECORDPROCESSINGERROR STRUCT<ERRORMESSAGE STRING, RECORD STRING, CAUSE ARRAY<STRING>>, PRODUCTIONERROR STRUCT<ERRORMESSAGE STRING>, SERIALIZATIONERROR STRUCT<TARGET STRING, ERRORMESSAGE STRING, RECORD STRING, CAUSE ARRAY<ST
RING>, `topic` STRING>, KAFKASTREAMSTHREADERROR STRUCT<ERRORMESSAGE STRING, THREADNAME STRING, CAUSE ARRAY<STRING>>>) WITH (KAFKA_TOPIC='default_ksql_processing_log', KEY_FORMAT='KAFKA', VALUE_FORMAT='JSON');

 Field   | Type


-----------------------------------------------------------------
```

```
ksql> SELECT * FROM production_changes
>WHERE created_at < '2023-04-14 12:00:00'
>EMIT CHANGES;
+-----------------------------------+-----------------------------------+-----------------------------------+-----------------------------------+-----------------------------------+-----------------------------------+
|ROWKEY                             |TITLE_ID                           |CHANGE_TYPE                        |BEFORE                             |AFTER                              |CREATED_AT                         |
+-----------------------------------+-----------------------------------+-----------------------------------+-----------------------------------+-----------------------------------+-----------------------------------+
|key2                               |2                                  |budget_update                      |{SEASON_ID=2, EPISODE_COUNT=8}     |{SEASON_ID=2, EPISODE_COUNT=8}     |2022-05-10 09:00:00                |
|key1                               |1                                  |season_length                      |{SEASON_ID=1, EPISODE_COUNT=12}    |{SEASON_ID=1, EPISODE_COUNT=8}     |2021-02-08 11:30:00                |
|key4                               |1                                  |season_length                      |{SEASON_ID=4, EPISODE_COUNT=8}     |{SEASON_ID=4, EPISODE_COUNT=9}     |2021-02-08 11:45:00                |
|key1                               |1                                  |season_length                      |{SEASON_ID=4, EPISODE_COUNT=12}    |{SEASON_ID=4, EPISODE_COUNT=8}     |2021-02-08 11:30:00                |
```

Q6 :

- Write a Push Query to return all records from production_changes where the column "change_type" starts with the word "season"

```
ksql> SELECT * FROM production_changes
>WHERE change_type LIKE 'season%'
>EMIT CHANGES;
+-----------------------------------+-----------------------------------+-----------------------------------+-----------------------------------+-----------------------------------+-----------------------------------+
|ROWKEY                             |TITLE_ID                           |CHANGE_TYPE                        |BEFORE                             |AFTER                              |CREATED_AT                         |
+-----------------------------------+-----------------------------------+-----------------------------------+-----------------------------------+-----------------------------------+-----------------------------------+
|key3                               |3                                  |season_renewal                     |{SEASON_ID=6, EPISODE_COUNT=0}     |{SEASON_ID=6, EPISODE_COUNT=5}     |2023-10-01 14:00:00                |
|key4                               |1                                  |season_length                      |{SEASON_ID=4, EPISODE_COUNT=8}     |{SEASON_ID=4, EPISODE_COUNT=9}     |2021-02-08 11:45:00                |
|key1                               |1                                  |season_length                      |{SEASON_ID=1, EPISODE_COUNT=12}    |{SEASON_ID=1, EPISODE_COUNT=8}     |2021-02-08 11:30:00                |
|key1                               |1                                  |season_length                      |{SEASON_ID=4, EPISODE_COUNT=12}    |{SEASON_ID=4, EPISODE_COUNT=8}     |2021-02-08 11:30:00                |
```

Q7:create a new stream to filter production_changes

- -> to read nested JSON objects

```
ksql> CREATE STREAM season_length_changes WITH (
>    KAFKA_TOPIC = 'season_length_changes',
>    PARTITIONS = 4,
>    REPLICAS = 1,
>    VALUE_FORMAT = 'JSON'
>) AS
>SELECT
>    rowkey,
>    title_id,
>    created_at,
>    COALESCE(after->season_id, before->season_id) AS season_id,
>    before->episode_count AS old_episode_count,
>    after->episode_count AS new_episode_count
>FROM production_changes
>WHERE change_type = 'season_length'
>EMIT CHANGES;

 Message
----------------------------------------------------
 Created query with ID CSAS_SEASON_LENGTH_CHANGES_7
----------------------------------------------------
ksql>
```

- result

```
ksql> SELECT * FROM season_length_changes EMIT CHANGES;
+-----------------------------------+-----------------------------------+-----------------------------------+-----------------------------------+-----------------------------------+-----------------------------------+
|ROWKEY                             |TITLE_ID                           |CREATED_AT                         |SEASON_ID                          |OLD_EPISODE_COUNT                  |NEW_EPISODE_COUNT                  |
+-----------------------------------+-----------------------------------+-----------------------------------+-----------------------------------+-----------------------------------+-----------------------------------+
|key99                              |1                                  |2024-01-01 10:00:00                |5                                  |10                                 |8                                  |
```

Q8:

- Write a Push Query to return the titles of all videos contained in season_length_changes

```
ksql> SELECT t.title
>FROM season_length_changes s
>INNER JOIN titles t ON s.title_id = t.id
>EMIT CHANGES;
+-----------------------------------------------------------------------
|TITLE
+-----------------------------------------------------------------------
|Stranger Things
```

Q9:Create the stream season_length_changes_enriched, which contains all attributes of season_length_changes, plus the title attribute from the titles collection

```
File  Edit  View  Terminal  Tabs  Help
ksql> CREATE STREAM season_length_changes_enriched WITH (
>    TIMESTAMP = 'created_at',
>    TIMESTAMP_FORMAT = 'yyyy-MM-dd HH:mm:ss'
>) AS
>SELECT
>    s.rowkey,
>    s.title_id,
>    s.created_at,
>    s.season_id,
>    s.old_episode_count,
>    s.new_episode_count,
>    t.title
>FROM season_length_changes s
>INNER JOIN titles t ON s.title_id = t.id
>EMIT CHANGES;

 Message
----------------------------------------------------------------
 Created query with ID CSAS_SEASON_LENGTH_CHANGES_ENRICHED_9
----------------------------------------------------------------
```

- result

```
ksql> SELECT * FROM season_length_changes_enriched EMIT CHANGES;
+-----------------------------+-----------------------------+-----------------------------+-----------------------------+-----------------------------+-----------------------------+-----------------------------+
|TITLE_ID                     |ROWKEY                       |CREATED_AT                   |SEASON_ID                    |OLD_EPISODE_COUNT            |NEW_EPISODE_COUNT            |TITLE                        |
+-----------------------------+-----------------------------+-----------------------------+-----------------------------+-----------------------------+-----------------------------+-----------------------------+
|1                            |key99                        |2024-01-01 10:00:00          |5                            |10                           |8                            |Stranger Things              |
```

Q10:

```
ksql> CREATE TABLE season_length_change_counts AS
>SELECT
>    title,
>    COUNT(*) AS total_changes,
>    LATEST_BY_OFFSET(new_episode_count) AS episode_count
>FROM season_length_changes_enriched
>WINDOW TUMBLING (SIZE 1 HOUR)
>GROUP BY title
>EMIT CHANGES;

 Message
----------------------------------------------------------------
 Created query with ID CTAS_SEASON_LENGTH_CHANGE_COUNTS_11
----------------------------------------------------------------
```

- result

```
ksql> SELECT * FROM season_length_change_counts;
+-----------------------------------------+-----------------------------------------+-----------------------------------------+-----------------------------------------+-----------------------------------------+
|TITLE                                    |WINDOWSTART                              |WINDOWEND                                |TOTAL_CHANGES                            |EPISODE_COUNT                            |
+-----------------------------------------+-----------------------------------------+-----------------------------------------+-----------------------------------------+-----------------------------------------+
|Stranger Things                          |1612782000000                            |1612785600000                            |3                                        |9                                        |
Query terminated
ksql>
```

EXo2 build a real-time data pipeline.

```
PROBLEMS 3   SQL HISTORY   DEBUG CONSOLE   TASK MONITOR   TERMINAL   PORTS   OUTPUT   SPELL CHECKER        java - Exo2

(venv) abdallah@abdallah-Laptop:~/2CS/S2/Big Data Tech/Labs/Lab7/Exo2$ spark-submit   --packages org.apache.spark:spark-sql-kafka-0-10_2.13:4.1.0
   ~/2CS/S2/Big\ Data\ Tech/Labs/Lab7/Exo2/exo2.py


-------------------------------------------
Batch: 1
-------------------------------------------
+-------------------+-------------------+--------+------------+
|window_start       |window_end         |title_id|change_count|
+-------------------+-------------------+--------+------------+
|2026-04-22 09:00:00|2026-04-22 10:00:00|1       |2           |
|2026-04-22 09:00:00|2026-04-22 10:00:00|3       |1           |
|2026-04-22 00:00:00|2026-04-22 01:00:00|1       |4           |
+-------------------+-------------------+--------+------------+

Launchpad   0 △ 3   Connect                                   Ln 51, Col 42   Spaces: 4   UTF-8   LF   {} Pyth
```

to test

```
INSERT INTO production_changes (rowkey, title_id, change_type, before,
after, created_at)
VALUES ('key11', 3, 'season_length',
        STRUCT(season_id:=4, episode_count:=8),
        STRUCT(season_id:=4, episode_count:=5),
        '2024-04-22 10:30:00');
```